

LẬP TRÌNH WEB NÂNG CAO

Thái Thanh Tùng
FITHOU

BÀI 5: Làm việc với Database

- ✓ Kết nối đến CSDL
- ✓ Cập nhật dữ liệu vào nguồn CSDL
- ✓ Thêm, Sửa, Xóa (CRUD)
- ✓ Sắp xếp, Tìm kiếm và Phân trang

Làm việc với Database

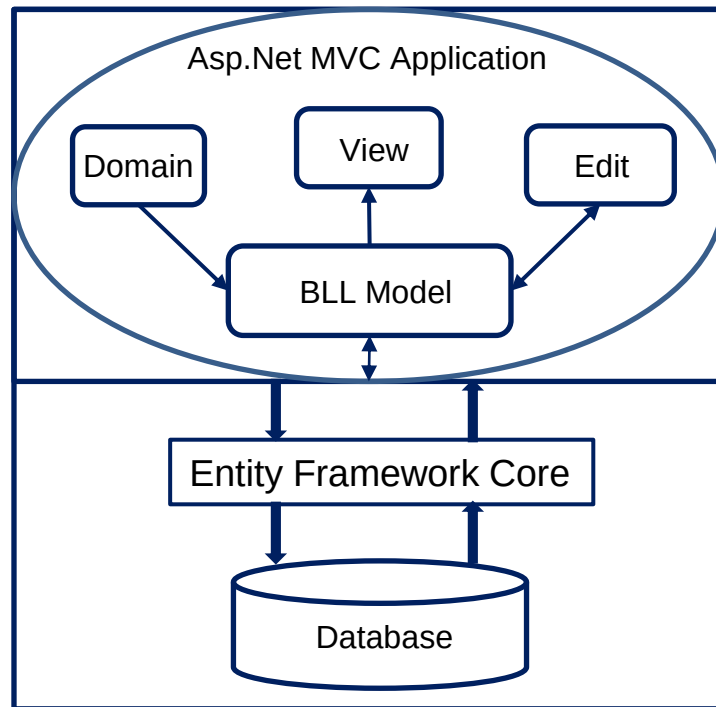
5. Làm việc với Database

- Ứng dụng Web cần phải sử dụng cơ sở dữ liệu thực sự để lưu trữ và truy xuất dữ liệu. ASP.NET core MVC Framework sử dụng công nghệ truy cập dữ liệu Entity Framework để thiết lập và làm việc với Model.
- EF ánh xạ đối tượng trong chương trình với bảng dữ liệu quan hệ giúp tự động hóa quá trình truy cập và lưu trữ dữ liệu trong cơ sở dữ liệu và ngược lại (ORM).
- EF có thể tự động tạo câu lệnh truy vấn SQL cần thiết để tạo cơ sở dữ liệu dựa trên các lớp Model hoặc xây dựng CRUD mô đun để truy vấn dữ liệu.
- Entity Framework Core, còn được gọi là EF Core, là phiên bản mới nhất của Entity Framework và được viết lại hoàn toàn từ đầu

Làm việc với Database

5. Làm việc với Database

- EF là một phần Data Layer, nằm giữa chương trình và CSDL, hỗ trợ việc tự động sinh mã cơ bản trong chương trình để truy cập dữ liệu tùy chỉnh mà chúng ta thường viết



Tạo kết nối tới CSDL nguồn

Truy xuất dữ liệu từ nguồn

Cập nhật trạng thái dữ liệu hiển thị

Thao tác truy vấn dữ liệu với nguồn

Đồng bộ trạng thái dữ liệu tại nguồn

Hình 5.1. Mô hình làm việc với CSDL

Làm việc với Database

5.1. Kết nối tới dữ liệu nguồn

Cài đặt EF Core: Framework làm việc với CSDL

- Tools > NuGet Package Manager > Package Manager Console
PM> Install-Package Microsoft.EntityFrameworkCore.SqlServer

Tạo DbContext: Lớp ngữ cảnh, tạo môi trường liên kết với dữ liệu nguồn.

- Lớp **ApplicationDbContext** xử lý nhiệm vụ kết nối với cơ sở dữ liệu và ánh xạ các đối tượng Model tới các bản ghi cơ sở dữ liệu.
- Lớp **ApplicationDbContext** được đăng ký với vùng chứa Dependency Injection của ConfigureServices trong tệp **program.cs** (hoặc startup.cs)

Configuration:

- Connectionstring trong **appsettings** (biến cục bộ)
- Environment variable (biến toàn cục)
- **Config** (tệp config)

Connect to DB: View > SQL Server Object Explorer.

Chú ý: Khi triển khai ứng dụng đến máy chủ development hoặc production, ta có thể sử dụng biến môi trường hoặc một phương pháp khác để đặt chuỗi kết nối tên Máy chủ SQL thực tế

Làm việc với Database

5.1. Kết nối tới dữ liệu nguồn

Tạo DbContext: Lớp ngữ cảnh, tạo môi trường liên kết với dữ liệu nguồn.

Tạo **ApplicationDbContext**: kết nối với cơ sở dữ liệu. Đăng ký **ApplicationDbContext** trong tệp **program.cs**

StudentDbContext.cs

```
1 using System;
2 using System.Collections.Generic;
3 using Microsoft.Extensions.Configuration;
4 using Microsoft.EntityFrameworkCore;
5 using Microsoft.EntityFrameworkCore.Metadata;
6 using System.Configuration;
7 using WebCoreMvcDemo.Models;
8
9 namespace WebCoreMvcDemo.Models
10 {
11     public partial class SchoolDbContext : DbContext
12     {
13         private readonly IConfiguration _configuration;
14         public SchoolDbContext(DbContextOptions<SchoolDbContext> options)
15             : base(options)
16         {
17         }
18
19         // Models
20         public virtual DbSet<Class> Classes { get; set; } = null!;
21         public virtual DbSet<Student> Students { get; set; } = null!;
22         public virtual DbSet<StudentClass> StudentClasses { get; set; } = null!;
23         public virtual DbSet<Teacher> Teachers { get; set; } = null!;
24
25         // CnnStr
26         protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
27         {
28             if (!optionsBuilder.IsConfigured)
29             {
30                 string myDbConnectionString = _configuration.GetConnectionString("DefaultConnection");
31                 optionsBuilder.UseSqlServer(myDbConnectionString);
32             }
33         }
34     }
35 }
```

program.cs

```
1 using Microsoft.EntityFrameworkCore;
2 using WebCoreMvcDemo.Data;
3 using WebCoreMvcDemo.Models;
4
5 var builder = WebApplication.CreateBuilder(args);
6
7 // Add services to the container.
8 builder.Services.AddControllersWithViews();
9
10 builder.Services.AddDbContext<SchoolDbContext>(options =>
11     options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")) ??
12     throw new InvalidOperationException("Không tìm thấy chuỗi liên kết 'SchoolDbContext'!"));
13
14 builder.Services.AddDbContext<StudentDbContext>(options =>
15     options.UseSqlServer(builder.Configuration.GetConnectionString("StudentConnection")) ??
16     throw new InvalidOperationException("Không tìm thấy chuỗi liên kết 'StudentDbContext'!"));
17
18 var app = builder.Build();
19
20 using (var scope = app.Services.CreateScope()) // Tạo service Dependency Injection
21 {
22     var services = scope.ServiceProvider;
23     ImportData.Initialize(services);
24 }
25
26 // Configure the HTTP request pipeline.
27 if (app.Environment.IsDevelopment())
28 {
29     app.UseExceptionHandler("/Home/Error");
30     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts
31     app.UseHsts();
32 }
33
34 app.UseHttpsRedirection();
35 app.UseStaticFiles();
36 }
```

An unhandled exception occurred while processing the request.

InvalidOperationException: Unable to resolve service for type 'WebCoreMvcDemo.Data.StudentDbContext' while attempting to activate 'WebCoreMvcDemo.Controllers.StudentValidationsController'.

Microsoft.Extensions.DependencyInjection.ActivatorUtilities.GetService(IServiceProvider sp, Type type, Type requiredBy, bool isDefaultParameterRequired)

Service Error

Chú ý: Chuỗi ConnectionString trong Configuration sẽ để lộ thông tin kết nối CSDL sử dụng Manage User Secret nhưng chú ý trong môi trường thực thi của WebCore Application

Làm việc với Database

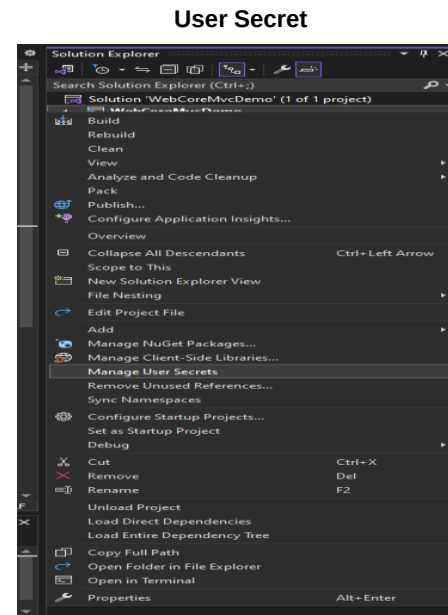
5.1. Kết nối tới dữ liệu nguồn

Configurations: ConnectionString, có thể sử dụng Manage User Secret để lưu ConnectionString

appsettings.json

```
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=DESKTOP-TUNG; Database=SchoolDB; User Id=sa; Password=tung; Trusted_Connection=True; MultipleActiveResultSets=true",
4     "StudentConnection": "Server=DESKTOP-TUNG; Database=StudentDB; User Id=sa; Password=tung; Trusted_Connection=True; MultipleActiveResultSets=true"
5   },
6   "Logging": {
7     "LogLevel": {
8       "Default": "Information",
9       "Microsoft.AspNetCore": "Warning"
10    }
11  },
12  "AllowedHosts": "*"
13 },
14
15
```

Config
ConnectionString



lauchSettings.json

```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*"
9 },
10
11
```

Development
Variables

Chú ý: Khi chuyển sang chế độ Production phải tích hợp cả Key value in Manage User Secret. Tham khảo ASP.NET core Environment Variables

Làm việc với Database

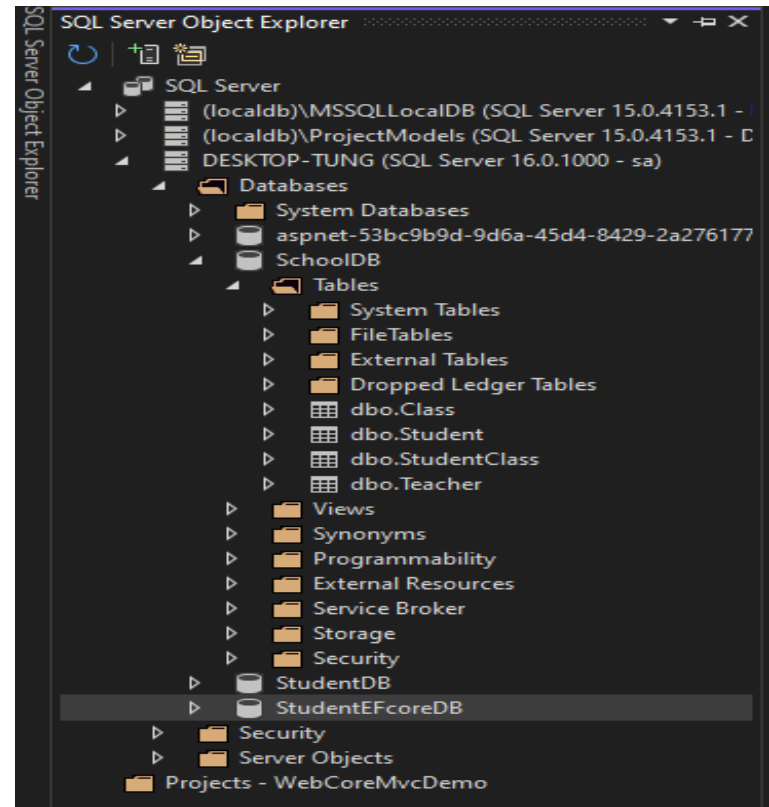
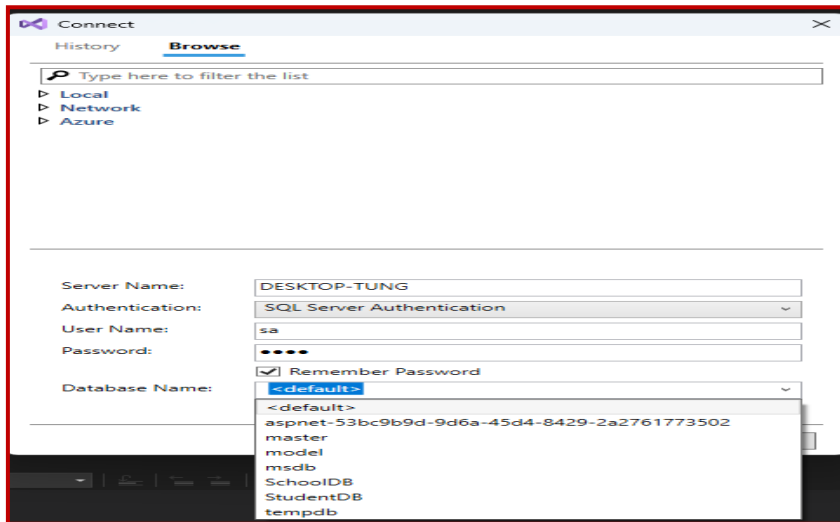
5.1. Kết nối tới dữ liệu nguồn

Khởi tạo SchoolDB

Database First: Thiết lập CSDL và cập nhật dữ liệu

- **Connect to DB:** View > SQL Server Object Explorer
- **Khởi tạo CSDL:** Tạo các thực thể (dbo.Tables)

Connect SQLServer



Làm việc với Database

5.2. Cập nhật dữ liệu vào nguồn

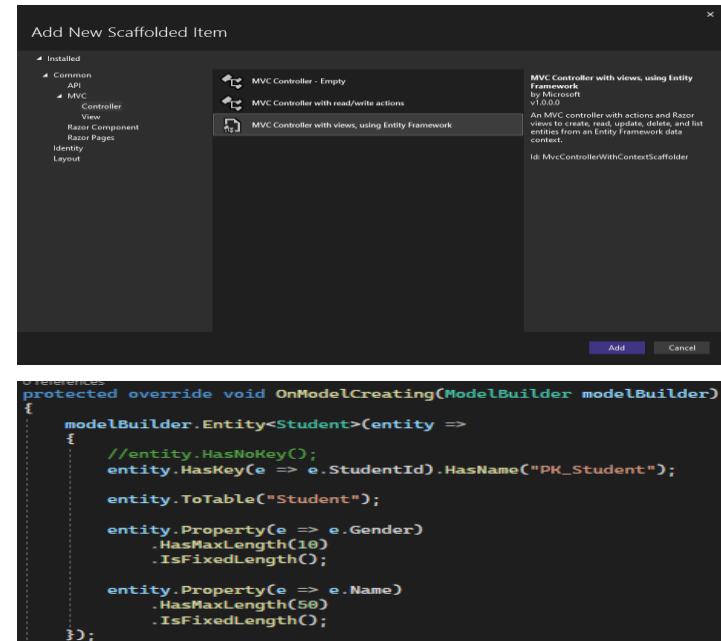
- Cập nhật dữ liệu vào nguồn: Tạo Model cập nhật dữ liệu **ImportData.cs** (SeedData).
- Tạo Controller sử dụng Entity Framework: **StudentEFController.cs**

ImportData.cs

```
using Microsoft.EntityFrameworkCore;
using WebCoreMvcDemo.Data;

namespace WebCoreMvcDemo.Models.SchoolDB
{
    public static class ImportData
    {
        public static void Initialize(IServiceProvider serviceProvider)
        {
            using (var context = new SchoolDbContext(
                serviceProvider.GetRequiredService<
                    DbContextOptions<SchoolDbContext>>())
            {
                if (context == null || context.Students == null)
                {
                    throw new ArgumentNullException("SchoolDbContext không tồn tại");
                }
                // Kiểm tra xem có dữ liệu không.
                if (context.Students.Any())
                {
                    return; // DB đã được khởi tạo dữ liệu.
                }
                context.Students.AddRange(
                    new Student
                    {
                        //StudentId = 1,
                        Name = "Nguyễn Thành Bình",
                        Gender = "Nam",
                        DOB = Convert.ToDateTime("5/15/2005"),
                        AccumulateGrade = 7.88M
                    },
                    new Student
                    {
                        //StudentId = 2,
                        Name = "Trần Phương Thảo",
                        Gender = "Nữ",
                        DOB = Convert.ToDateTime("8/25/2004"),
                        AccumulateGrade = 8.35M
                    },
                    new Student
                    {
                        //StudentId = 3,
                        Name = "Lê Minh Ngọc",
                        Gender = "Nam",
                        DOB = Convert.ToDateTime("11/20/2005"),
                        AccumulateGrade = 7.63M
                    }
                );
                context.SaveChanges();
            }
        }
    }
}
```

Tạo Controller sử dụng Entity Framework

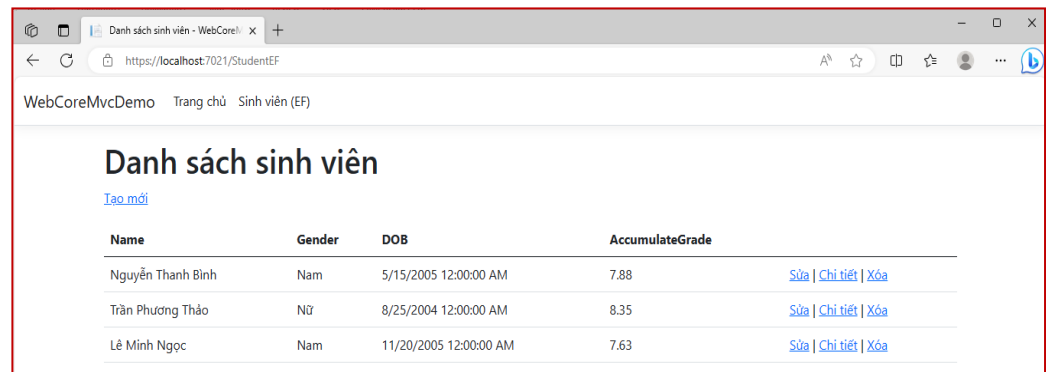
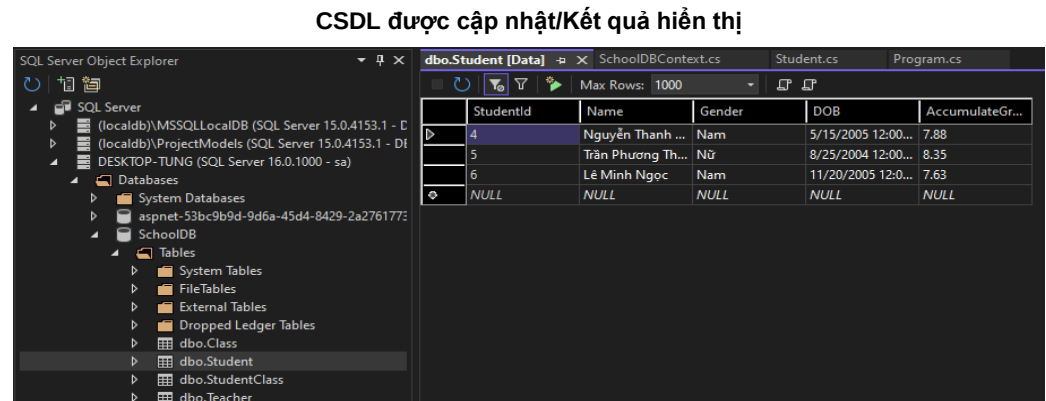
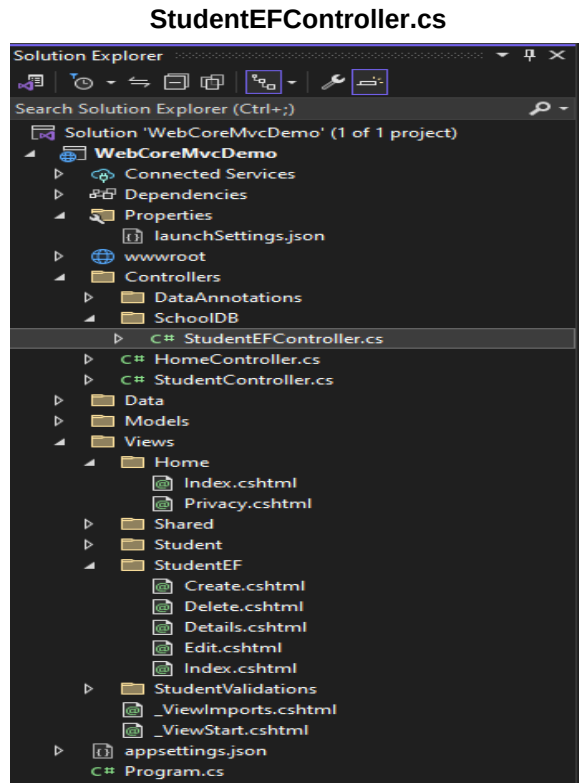


Chú ý: Tên thực thể + Id sẽ được EF coi là khóa chính. Nếu không tạo khóa cần phải gọi `HasNoKey()` trong `DbContext`

Làm việc với Database

5.2. Cập nhật dữ liệu vào nguồn

- Kết quả thực thi tạo dữ liệu cho nguồn sử dụng Entity Framework và hiển thị kết quả



Database Access

5.3. Data Relations

Code-First và Data Relations

One to One:

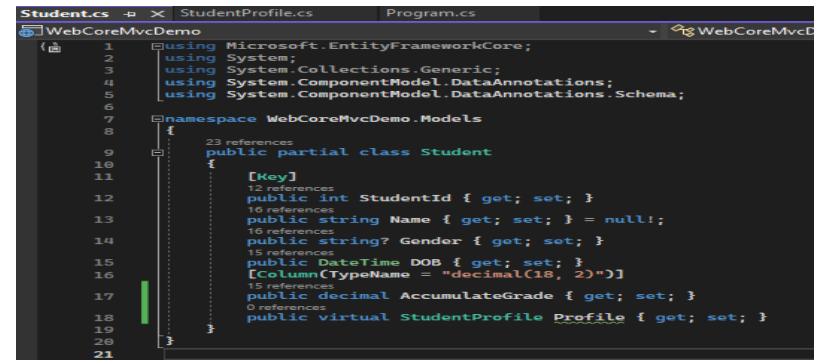
- Thiết lập quan hệ giữa hai thực thể bằng cách sử dụng cả **DataAnnotation** khóa ngoại và **FluentAPI** **[ForeignKey]** và **[InverseProperty]**

- Cấu hình quan hệ giữa hai thực thể bằng cách sử dụng **FluentAPI**.

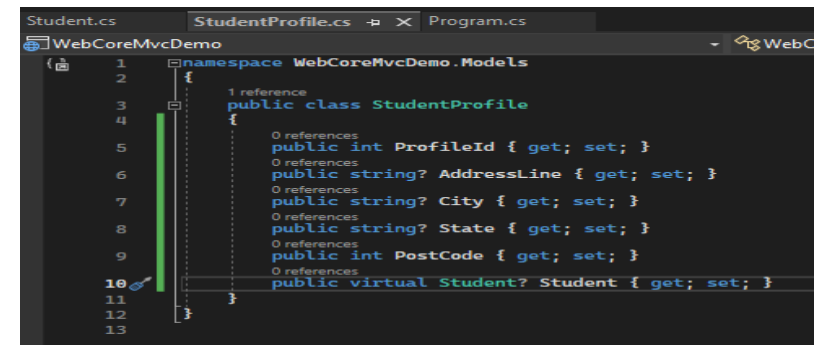
**HasOne(), HasMany(),
WithOne, WithMany,
HasForeignKey, IsRequired()**

- Tạo khóa ngoại, Inverse Property:
[public virtual Entity]

Chú ý: Quan hệ 1-1 hoặc 1-0, một hàng trong Bảng A có thể có tối đa một hoặc 0 hàng trùng khớp trong Bảng B và ngược lại.



```
1 using Microsoft.EntityFrameworkCore;
2 using System;
3 using System.Collections.Generic;
4 using System.ComponentModel.DataAnnotations;
5 using System.ComponentModel.DataAnnotations.Schema;
6
7 namespace WebCoreMvcDemo.Models
8 {
9     23 references
10     public partial class Student
11     {
12         [Key]
13         12 references
14         public int StudentId { get; set; }
15         10 references
16         public string Name { get; set; } = null!;
17         16 references
18         public string? Gender { get; set; }
19         15 references
20         public DateTime DOB { get; set; }
21         15 references
22         public decimal AccumulateGrade { get; set; }
23         0 references
24         public virtual StudentProfile Profile { get; set; }
25     }
26 }
```



```
1 namespace WebCoreMvcDemo.Models
2 {
3     1 reference
4     public class StudentProfile
5     {
6         0 references
7         public int ProfileId { get; set; }
8         0 references
9         public string? AddressLine { get; set; }
10         0 references
11         public string? City { get; set; }
12         0 references
13         public string? State { get; set; }
14         0 references
15         public int PostCode { get; set; }
16         0 references
17         public virtual Student? Student { get; set; }
18     }
19 }
```

Student / Student Profile

Database Access

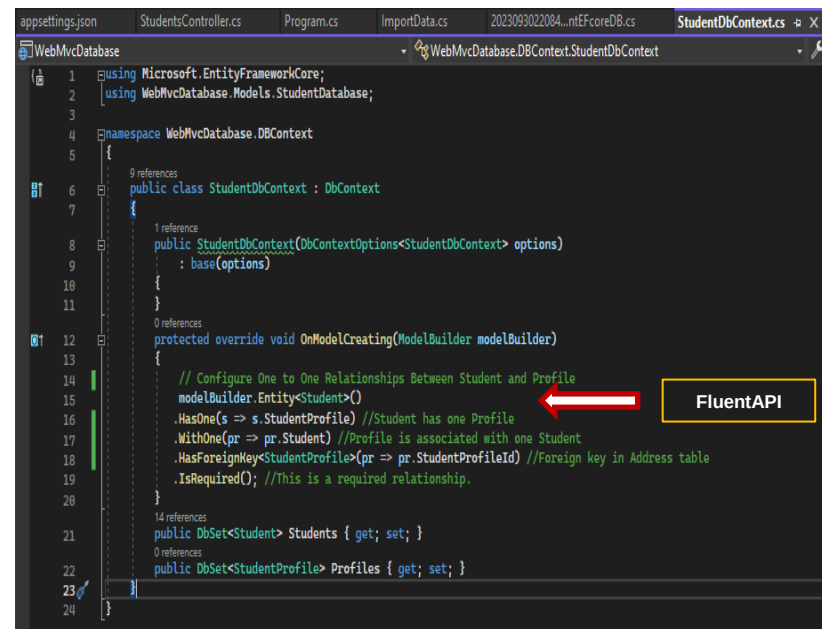
5.3. Data Relations

Code-First và Data Relations

One to One:

- Tạo DbContext kết nối SQLServer
- Cấu hình quan hệ 1-1 giữa hai thực thể bằng cách sử dụng FluentAPI:
 - **HasOne()**: Có 1 bảng quan hệ 1-1
 - **WithOne**: Quan hệ ngược 1-1
 - **HasForeignKey**: Khóa ngoại
 - **IsRequired()**: Bắt buộc

OnModelCreating trong DbContext



```
1 using Microsoft.EntityFrameworkCore;
2 using WebMvcDatabase.Models.StudentDatabase;
3
4 namespace WebMvcDatabase.DbContext
5 {
6     public class StudentDbContext : DbContext
7     {
8         public StudentDbContext(DbContextOptions<StudentDbContext> options)
9             : base(options)
10         {
11         }
12
13         protected override void OnModelCreating(ModelBuilder modelBuilder)
14         {
15             // Configure One to One Relationships Between Student and Profile
16             modelBuilder.Entity<Student>()
17                 .HasOne(s => s.StudentProfile) //Student has one Profile
18                 .WithOne(pr => pr.Student) //Profile is associated with one Student
19                 .HasForeignKey<StudentProfile>(pr => pr.StudentProfileId) //Foreign key in Address table
20                 .IsRequired(); //This is a required relationship.
21
22             public DbSet<Student> Students { get; set; }
23             public DbSet<StudentProfile> Profiles { get; set; }
24         }
25     }
```

Chú ý: Nếu quan hệ không bắt buộc thì sử dụng **IsRequired(false)**

Database Access

5.3. Data Relations

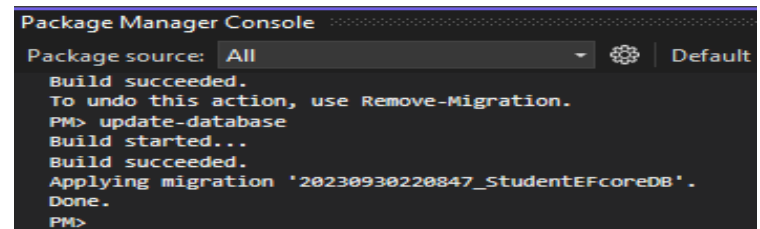
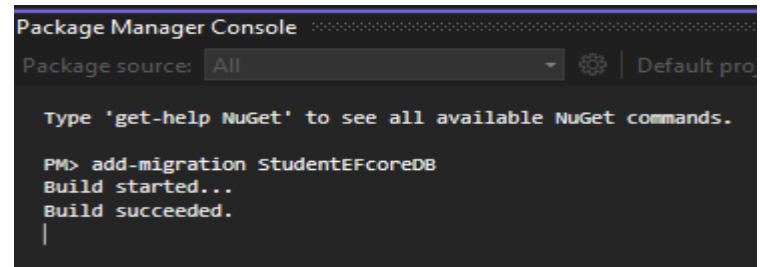
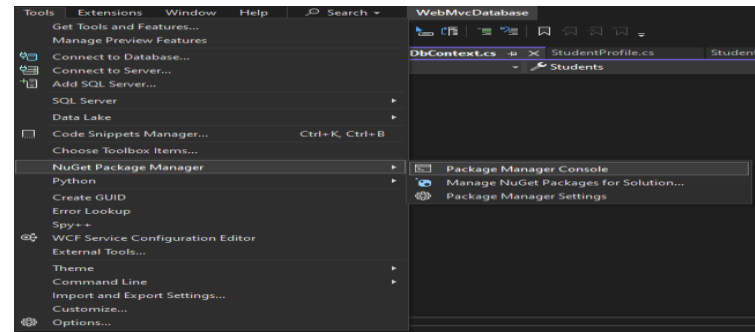
Code-First và Migration

Migrate cấu trúc tạo CSDL:

- Sử dụng (VS) Package Manager Console:

- PM> add-migration DBName
- PM> update-database

Chú ý: Để rollback quá trình Migration sử dụng
remove-migration



Package Manager Console

Database Access

5.3. Data Relations

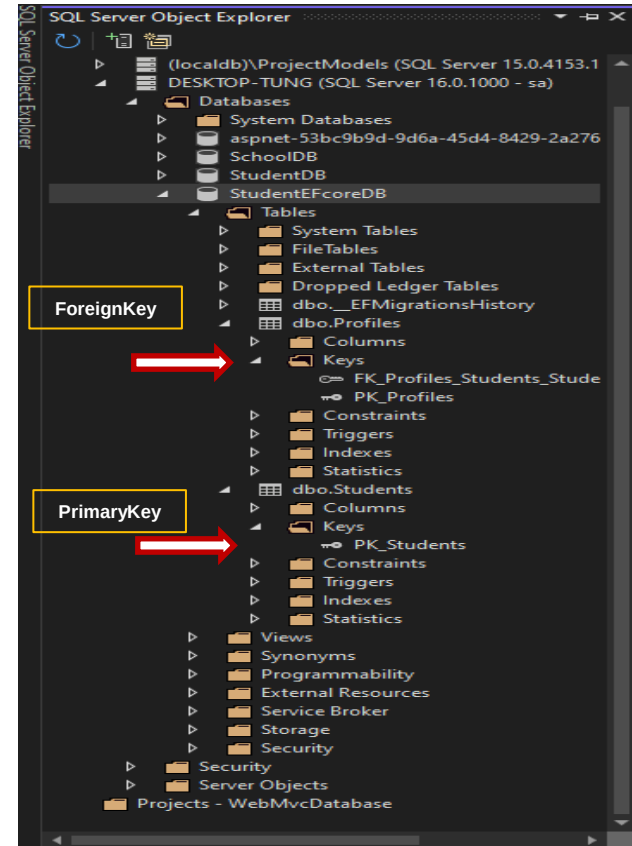
Code-First và Migration

Migrate cấu trúc tạo CSDL:

- Sử dụng (VS) Package Manager Console:

- PM> add-migration DBName
- PM> update-database

Chú ý: Để rollback quá trình Migration sử dụng
remove-migration



SQLServer: 1-1 Relationship

Database Access

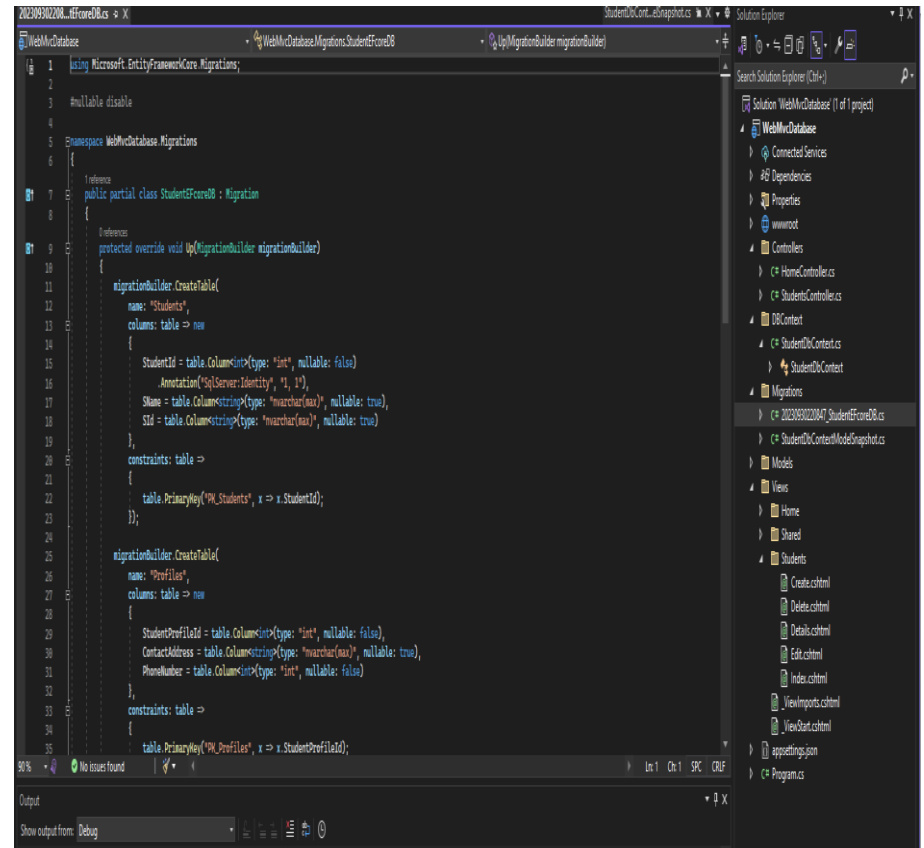
5.3. Data Relations

Code-First và Migration

Migration Folder:

- Key_DBName.cs
- DBNameContextModel.cs

Chú ý: Để rollback quá trình Migration sử dụng
remove-migration



Migration Folder

Database Access

5.3. Data Relations

Code-First và ImportData

ImportData.cs:

- Tạo DbContext kết nối CSDL
- Khởi tạo dữ liệu Context.DbSet
- SaveChanges() lưu kết quả

```
using Microsoft.EntityFrameworkCore;
using System.Net;
using WebMvcDatabase.DbContext;
using WebMvcDatabase.Models.StudentDatabase;

namespace WebMvcDatabase.Models
{
    public class ImportData
    {
        public static void Initialize(IServiceProvider serviceProvider)
        {
            //Creating an Instance of the Context Class
            using (var context = new StudentDbContext(
                serviceProvider.GetRequiredService<
                    DbContextOptions<StudentDbContext>>()
            ))
            {
                if (context == null || context.Students == null)
                {
                    throw new ArgumentNullException("Null Context");
                }

                var studentProfile = new StudentProfile() { ContactAddress = "Hà
                Nội", PhoneNumber = 123456 };

                //Creating Student Entity, configuring the Related Student Profile
                Entity
                var student = new Student() { SName = "Nguyễn Ihanh Binh", Sid =
                "MSV123456", StudentProfile = studentProfile };
                context.Students.Add(student);
                context.SaveChanges();
            }
        }
    }
}
```

```
using Microsoft.EntityFrameworkCore;
using System.Net;
using WebMvcDatabase.DbContext;
using WebMvcDatabase.Models.StudentDatabase;

namespace WebMvcDatabase.Models
{
    public class ImportData
    {
        public static void Initialize(IServiceProvider serviceProvider)
        {
            //Creating an Instance of the Context Class
            using (var context = new StudentDbContext(
                serviceProvider.GetRequiredService<
                    DbContextOptions<StudentDbContext>>()
            ))
            {
                if (context == null || context.Students == null)
                {
                    throw new ArgumentNullException("Null Context");
                }

                var classes = new List<Class>
                {
                    new Class() { ClassName = "C#"},
                    new Class() { ClassName = "SQL Server"},
                    new Class() { ClassName = "ASP.NET core MVC"},
                }

                //Creating Student Entity, configuring the Related Student Class Entity
                var student = new Student()
                {
                    SName = "Nguyễn Ihanh Binh", Sid = "MSV123456",
                    Class = classes
                };
                context.Students.Add(student);
                context.SaveChanges();
            }
        }
    }
}
```

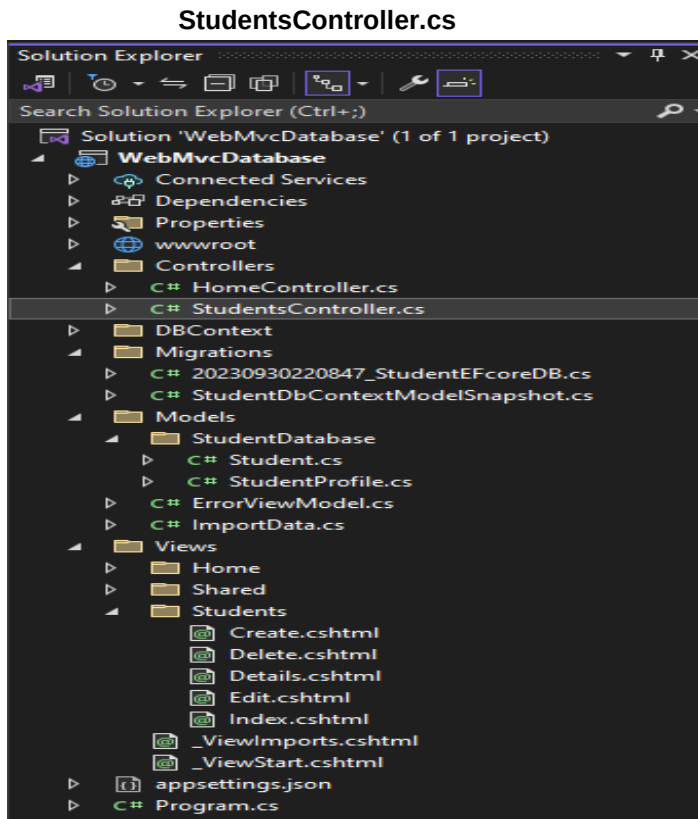
Chú ý: Quan hệ 1-n và n-n thì sử dụng List<Entity>

ImportData.cs

Làm việc với Database

5.3. Data Relations

- Kết quả thực thi tạo dữ liệu cho nguồn sử dụng Entity Framework và hiển thị kết quả



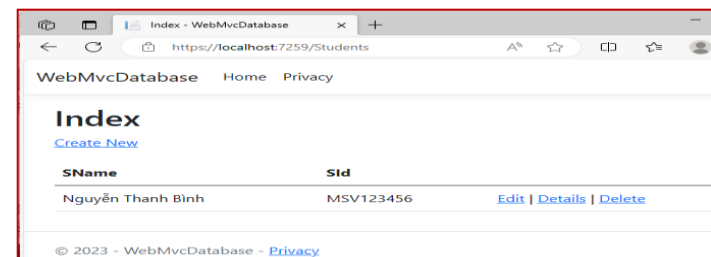
CSDL được cập nhật/Kết quả hiển thị

dbo.Students [Data] ImportData.cs 2023093022

	StudentId	SName	SId
▶	1	Nguyễn Thanh ...	MSV123456
⚙	NULL	NULL	NULL

dbo.Profiles [Data] dbo.Students [Data] ImportData.cs

	StudentProfileId	ContactAddress	PhoneNumber
▶	1	Hà Nội	123456
⚙	NULL	NULL	NULL



Database Access

5.4. Thao tác với dữ liệu

CRUD Operation: Create, Read, Update, Delete

Ví dụ: khi HTTP GET request được gửi đi

<https://localhost:7259/Students/Edit/1>

Controller sẽ gọi các phương thức **onGetAsync** và **onPostAsync** để thực hiện request.

- Phương thức **[HttpGet] public Async Task<ActionResult>** tìm Student từ cơ sở dữ liệu và trả về Edit.cshtml
- Razor Page Edit.cshtml hiển thị dữ liệu và gọi HTTP POST request để cập nhật dữ liệu đã chỉnh sửa
- **[HttpPost] Public Async Task <ActionResult>** sẽ cập nhật dữ liệu về nguồn

```
// GET: Students/Edit/5
0 references
public async Task<ActionResult> Edit(int? id)
{
    if (id == null || _context.Students == null)
    {
        return NotFound();
    }

    var student = await _context.Students.FindAsync(id);
    if (student == null)
    {
        return NotFound();
    }

    return View(student);
}
```

```
// POST: Students/Edit/5
// To protect from overposting attacks, enable the specific properties you want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Edit(int id, [Bind("StudentId,SName,SId")] Student student)
{
    if (id != student.StudentId)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            _context.Update(student);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!StudentExists(student.StudentId))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }

        return RedirectToAction(nameof(Index));
    }
    return View(student);
}
```

Database Access

5.4. Thao tác với dữ liệu

Sắp xếp, Lọc, Phân trang dữ liệu

StudentController.cs

- Tạo QueryString sắp xếp: **ViewData["Name"]**
- Truy vấn dữ liệu (Linq): lấy dữ liệu từ **Students**
- Thiết lập sắp xếp: **students.OrderBy()**

```
public async Task<IActionResult> Index(string sortOrder)
{
    ViewData["NameSortParm"] = String.IsNullOrEmpty(sortOrder) ? "name_desc" : "";

    var students = from s in _context.Students
                   select s;

    switch (sortOrder)
    {
        case "name_desc":
            students = students.OrderByDescending(s => s.SName);
            break;
        default:
            students = students.OrderBy(s => s.SId);
            break;
    }

    return View(await students.AsNoTracking().ToListAsync());
}
```

Index.cshtml

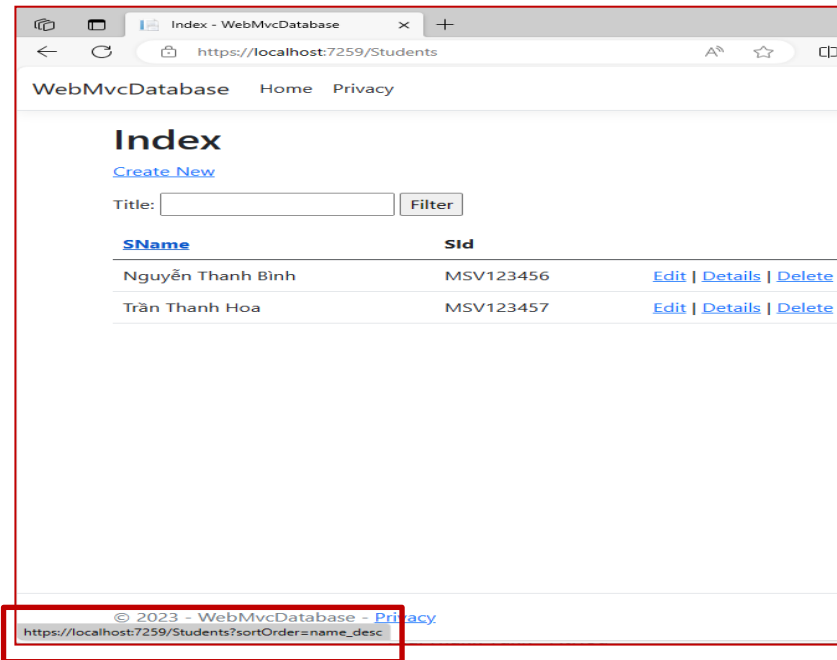
- Đặt link request truyền QueryString

```
<th>
  <a asp-action="Index" asp-route-sortOrder="@ViewData["NameSortParm"]">@Html.DisplayNameFor(model => model.SName)</a>
</th>
```

Database Access

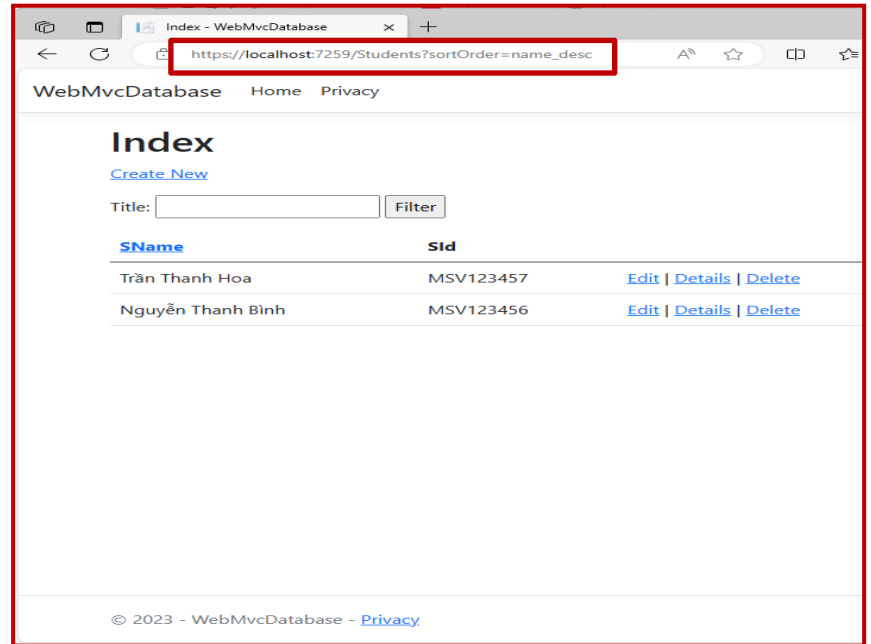
5.4. Thao tác với dữ liệu

Sắp xếp dữ liệu



The screenshot shows the 'Index' page of the 'WebMvcDatabase' application. The browser address bar displays 'https://localhost:7259/Students'. The page includes a 'Create New' link, a 'Title' input field, and a 'Filter' button. Below these, there is a table with two columns: 'SName' and 'Sid'. The table contains two rows of student data. At the bottom of the page, the footer shows '© 2023 - WebMvcDatabase - Privacy' and the URL 'https://localhost:7259/Students?sortOrder=name_desc'.

SName	Sid
Nguyễn Thanh Bình	MSV123456
Trần Thanh Hoa	MSV123457



The screenshot shows the 'Index' page of the 'WebMvcDatabase' application. The browser address bar displays 'https://localhost:7259/Students?sortOrder=name_desc', which is highlighted with a red box. The page layout is identical to the previous screenshot, but the table data is sorted by name in descending order. The footer shows '© 2023 - WebMvcDatabase - Privacy'.

SName	Sid
Trần Thanh Hoa	MSV123457
Nguyễn Thanh Bình	MSV123456

Database Access

5.4. Thao tác với dữ liệu

Lọc – Tìm kiếm

StudentController.cs

- Tạo QueryString tìm kiếm: CurrentFilter
- Truy vấn dữ liệu (Linq)

```
public async Task<IActionResult> Index(string sortOrder, string searchQuery)
{
    ViewData["NameSortParm"] = String.IsNullOrEmpty(sortOrder) ? "name_desc" : "";
    ViewData["CurrentFilter"] = searchQuery;

    var students = from s in _context.Students
                   select s;

    if (!String.IsNullOrEmpty(searchQuery))
    {
        students = students.Where(s => s.SName.Contains(searchQuery)
                                   || s.SId.Contains(searchQuery));
    }
}
```

Index.cshtml

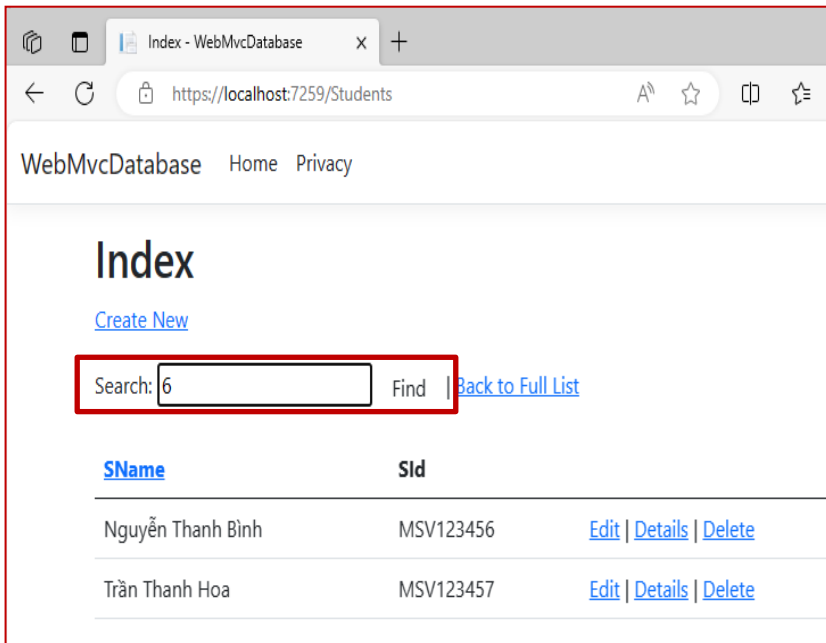
- Tạo form tìm kiếm truyền CurrentFilter

```
<form asp-action="Index" method="get">
    <div class="form-actions no-color">
        <p>
            Search: <input type="text" name="SearchQuery" value="@ViewData["CurrentFilter"]" />
            <input type="submit" value="Find" class="btn btn-default" /> |
            <a asp-action="Index">Back to Full List</a>
        </p>
    </div>
</form>
```

Database Access

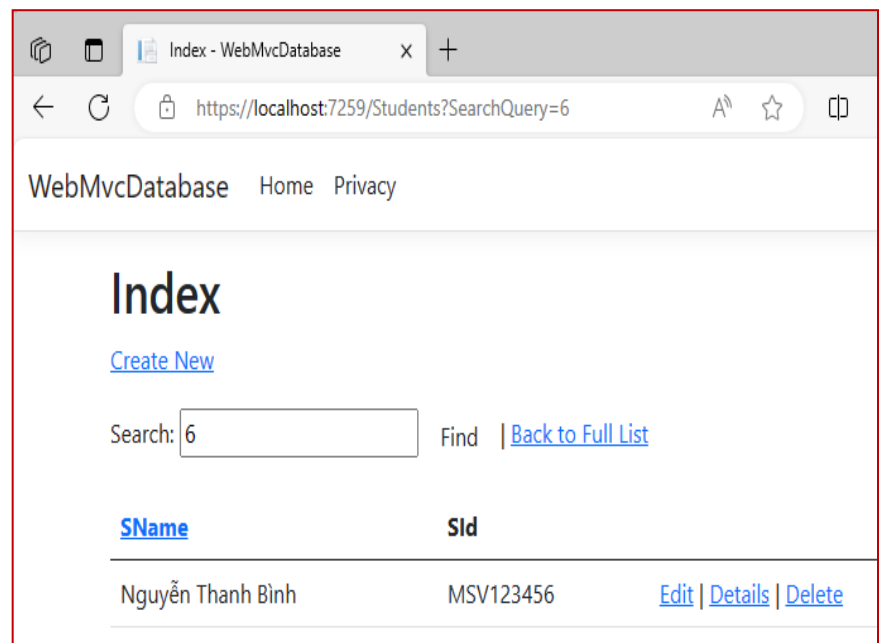
5.4. Thao tác với dữ liệu

Tìm kiếm dữ liệu



The screenshot shows the 'Index' page of the 'WebMvcDatabase' application. The browser address bar is 'https://localhost:7259/Students'. The page has a navigation bar with 'Home' and 'Privacy' links. Below the 'Index' heading is a 'Create New' link. A search bar contains the text '6', followed by a 'Find' button and a 'Back to Full List' link. Below the search bar is a table with two columns: 'SName' and 'Sid'. The table contains two rows of student data.

SName	Sid
Nguyễn Thanh Bình	MSV123456
Trần Thanh Hoa	MSV123457



The screenshot shows the 'Index' page of the 'WebMvcDatabase' application after a search. The browser address bar is 'https://localhost:7259/Students?SearchQuery=6'. The page layout is identical to the previous screenshot, but the table now only displays one row of student data, filtered by the search query '6'.

SName	Sid
Nguyễn Thanh Bình	MSV123456

Database Access

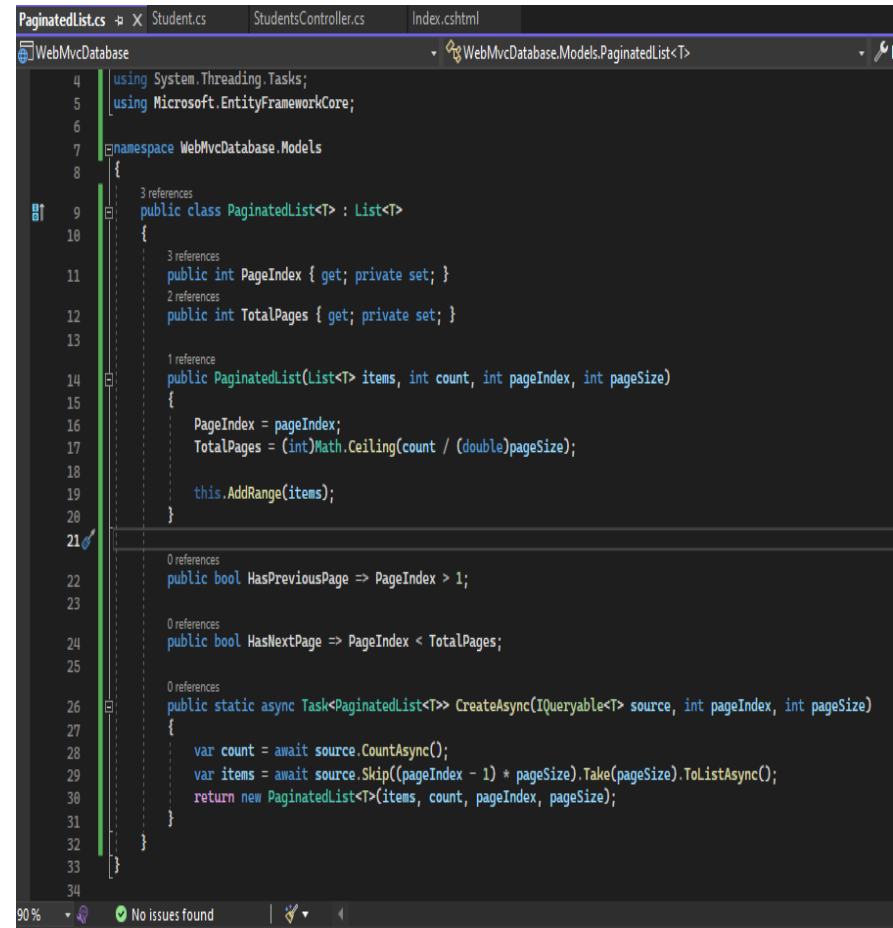
5.4. Thao tác với dữ liệu

Phân trang dữ liệu

Để phân trang vào trang danh mục Students, ta tạo một lớp **PaginatedList.cs**

- PageIndex: xác định chỉ số trang hiện hành
- TotalPage: tổng số bản ghi từ nguồn
- Tạo List cập nhật dữ liệu theo trang truy vấn hiện hành
 - Skip: bỏ qua các bản ghi
 - Take: Truy vấn số lượng bản ghi theo trang

Lớp PaginatedList sẽ truy vấn từng trang dữ liệu thay vì luôn truy xuất tất cả các bản ghi trong bảng dữ liệu nguồn



```
4 using System.Threading.Tasks;
5 using Microsoft.EntityFrameworkCore;
6
7 namespace WebMvcDatabase.Models
8 {
9     public class PaginatedList<T> : List<T>
10     {
11         public int PageIndex { get; private set; }
12         public int TotalPages { get; private set; }
13
14         public PaginatedList(List<T> items, int count, int pageIndex, int pageSize)
15         {
16             PageIndex = pageIndex;
17             TotalPages = (int) Math.Ceiling(count / (double) pageSize);
18
19             this.AddRange(items);
20         }
21
22         public bool HasPreviousPage => PageIndex > 1;
23
24         public bool HasNextPage => PageIndex < TotalPages;
25
26         public static async Task<PaginatedList<T>> CreateAsync(IQueryable<T> source, int pageIndex, int pageSize)
27         {
28             var count = await source.CountAsync();
29             var items = await source.Skip((pageIndex - 1) * pageSize).Take(pageSize).ToListAsync();
30             return new PaginatedList<T>(items, count, pageIndex, pageSize);
31         }
32     }
33 }
```

Database Access

5.4. Thao tác với dữ liệu

StudentController.cs

```
// GET: Students
public async Task<ActionResult> Index(string sortOrder, string currentFilter,
string searchQuery, int? pageNumber)
{
    ViewData["NameSortParam"] = String.IsNullOrEmpty(sortOrder) ? "name_desc" : "";

    if (searchQuery != null)
    {
        pageNumber = 1;
    }
    else
    {
        searchQuery = currentFilter;
    }

    ViewData["CurrentFilter"] = searchQuery;
    var students = from s in _context.Students
        select s;

    if (!String.IsNullOrEmpty(searchQuery))
    {
        students = students.Where(s => s.SName.Contains(searchQuery)
            || s.SId.Contains(searchQuery));
    }

    switch (sortOrder)
    {
        case "name_desc":
            students = students.OrderByDescending(s => s.SName);
            break;
        default:
            students = students.OrderBy(s => s.SId);
            break;
    }

    int pageSize = 3;
    return View(await PaginatedList<Student>.CreateAsync(students.AsNoTracking(),
        pageNumber ?? 1, pageSize));
}
```

Index.cshtml

```
@{
    var prevDisabled = !Model.HasPreviousPage ? "disabled" : "";
    var nextDisabled = !Model.HasNextPage ? "disabled" : "";
}

<a asp-action="Index"
    asp-route-sortOrder="@ViewData["CurrentSort"]"
    asp-route-pageNumber="@((Model.PageIndex - 1))"
    asp-route-currentFilter="@ViewData["CurrentFilter"]"
    class="btn btn-default @prevDisabled">
    Previous
</a>

<a asp-action="Index"
    asp-route-sortOrder="@ViewData["CurrentSort"]"
    asp-route-pageNumber="@((Model.PageIndex + 1))"
    asp-route-currentFilter="@ViewData["CurrentFilter"]"
    class="btn btn-default @nextDisabled">
    Next
</a>
```


Database Access

5.4. Thao tác với dữ liệu

Phân trang dữ liệu

WebMvcDatabase Home Privacy

Index

[Create New](#)

Find by name: Search | [Back to Full List](#)

Last Name	Student Name	
Nguyễn Thanh Bình	MSV123456	Edit Details Delete
Trần Thanh Hoa	MSV123457	Edit Details Delete
Lê Văn Mạnh	MSV123458	Edit Details Delete

Previous **Next**

© 2023 - WebMvcDatabase - [Privacy](#)
<https://localhost:7259/Students?pageNumber=2>

WebMvcDatabase Home Privacy

Index

[Create New](#)

Find by name: Search | [Back to Full List](#)

Last Name	Student Name	
Phạm Thanh Hương	MSV123459	Edit Details Delete

Previous Next

KẾT THÚC

Q & A

XIN CHÀO VÀ HẸN GẶP LẠI TUẦN SAU